

Step 2 — A Real SQLite-Backed Store, and the Search Tool — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# db.py
import sqlite3
from datetime import datetime

def init_db(path):
    conn = sqlite3.connect(path)
    conn.execute(
        "CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY AUTOINCREMENT,
        )"
    )
    conn.commit()
    conn.close()

def insert_note(path, text):
    conn = sqlite3.connect(path)
    cursor = conn.execute(
        "INSERT INTO notes (text, created_at) VALUES (?, ?)", (text, datetime.now())
    )
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()
    return new_id

def find_notes(path, query):
    conn = sqlite3.connect(path)
    rows = conn.execute("SELECT text FROM notes WHERE text LIKE ?", (f"%{query}%"
```

```
conn.close()
return [row[0] for row in rows]
```

```
# server.py (add_note and search_notes only)
@mcp.tool()
def add_note(text):
    if not text.strip():
        raise Exception("Note text cannot be empty.")
    new_id = insert_note(DB_PATH, text)
    return f"Note added with id {new_id}."

@mcp.tool()
def search_notes(query):
    return find_notes(DB_PATH, query)
```

This works and survives a restart. It's missing type hints, a real custom exception (using bare `Exception` instead of something named), and its search is case-sensitive — `find_notes("Dentist")` won't match a note containing `"dentist"`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, a real custom exception, case-insensitive search

```
# exceptions.py
class EmptyNoteError(Exception):
    pass
```

```
# db.py
import sqlite3
from datetime import datetime

def init_db(path: str) -> None:
    conn = sqlite3.connect(path)
    conn.execute(
        """CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            text TEXT NOT NULL,
            created_at TEXT NOT NULL
        )"""
    )
    conn.commit()
    conn.close()

def insert_note(path: str, text: str) -> int:
    conn = sqlite3.connect(path)
    cursor = conn.execute(
        "INSERT INTO notes (text, created_at) VALUES (?, ?)",
        (text, datetime.now().isoformat()),
    )
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()
    return new_id

def find_notes(path: str, query: str) -> list[str]:
    conn = sqlite3.connect(path)
```

```

rows = conn.execute(
    "SELECT text FROM notes WHERE LOWER(text) LIKE ?",
    (f"%{query.lower()}%",),
).fetchall()
conn.close()
return [row[0] for row in rows]

```

```

# server.py (add_note and search_notes only)
from exceptions import EmptyNoteError
from db import init_db, insert_note, find_notes

DB_PATH = config.notes_db_path
init_db(DB_PATH)

@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note to the local notes store. Rejects empty or
    whitespace-only text with a clear error instead of storing it."""
    cleaned = text.strip()
    if not cleaned:
        raise EmptyNoteError("Note text cannot be empty or whitespace-only.")
    new_id = insert_note(DB_PATH, cleaned)
    logger.info("Note %d added", new_id)
    return f"Note added with id {new_id}."

@mcp.tool()
def search_notes(query: str) -> list[str]:
    """Search stored notes for a case-insensitive substring match."""
    return find_notes(DB_PATH, query)

```

Expected behavior: `add_note("Buy milk and eggs")` → `"Note added with id 1."`. Restart the server process entirely, then `search_notes("milk")` → `["Buy milk and eggs"]` — proving persistence actually works, not just that the code runs once. `search_notes("Dentist")` now matches a note containing `"dentist"`, because both sides are lowercased before comparing.

Difference from Basic: a named `EmptyNoteError` instead of a bare `Exception` (so `except EmptyNoteError:` elsewhere in the codebase can catch *specifically* this failure, not accidentally swallow every other kind of error too), full type hints, and a case-insensitive search — the same `.lower()` habit as Doc02's JSON-handling lessons about not trusting exact-case matches from the outside world.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — same code, plus proof of what happens when `EmptyNoteError` is raised

The `server.py` above already raises `EmptyNoteError` correctly. What's worth understanding is what happens next: the official `mcp` SDK's `FastMCP` catches any exception a `@mcp.tool()` - decorated function raises and turns it into a tool result with `isError=True`, carrying the exception's message as the error content — the client sees a clean, readable failure, not a crashed server process and not a raw Python traceback. This is the MCP-layer version of Doc06's "getting a tool's failure back to the model, correctly" section — the same idea (never let a tool's failure crash the whole program; hand back a clear result instead), just implemented for you by the SDK instead of something you write a `try / except` for by hand, the way Doc06's raw-API examples do.

```
# proof, run once from the Inspector or a client script:
# call_tool("add_note", {"text": ""})
# call_tool("add_note", {"text": "  "})
# both should come back as an error result containing:
# "Note text cannot be empty or whitespace-only."
# -- NOT a crashed server, NOT a silently-created blank note.
```

Approach 2 — an empty store handled as a normal case, not a failure

```
def find_notes(path: str, query: str) -> list[str]:
    conn = sqlite3.connect(path)
    rows = conn.execute(
        "SELECT text FROM notes WHERE LOWER(text) LIKE ?",
        (f"%{query.lower()}%",),
    ).fetchall()
    conn.close()
    return [row[0] for row in rows]
# an empty `rows` list here is a completely valid outcome --
# nothing above needs a special case for "no notes exist yet"
```

Expected behavior: `search_notes("anything")` against a freshly created, never-written-to database returns `[]` — no exception, no special-cased error message. A search finding nothing is not the same kind of failure as an empty *write*; only `add_note` needed a validation check, because only `add_note` was at risk of storing something invalid.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate gets the validation and the search working correctly. Approach 1 explains *why* raising `EmptyNoteError` is already enough — FastMCP's own error handling does the "hand it back as data, don't crash" step Doc06 taught you to write by hand for the raw API. Approach 2 isn't a code change at all; it's confirming, on purpose, that the one edge case which *doesn't* need special handling (an empty search result) is already handled correctly, by doing nothing extra.

Which one should you actually write? Intermediate's version, exactly as shown, is what belongs in this project's `server.py` and `db.py`. The two Advanced approaches aren't extra code to add — they're verification you should actually run once: call `add_note` with empty/whitespace text and confirm the client sees a clean error, and call `search_notes` against an empty database and confirm it returns `[]` without complaint. Both are 30-second checks that catch the two most common bugs in a step like this — a validation check that doesn't actually fire, and a "no results" case mistaken for a crash.